

(Near-)Pareto-Optimal Synthesized OS-Level Scheduling for Serverless Systems

Yuxuan Zhao
Leiden University
Leiden, Netherlands
y.zhao@liacs.leidenuniv.nl

Zhao Yang
VU Amsterdam
Amsterdam, Netherlands
z.yang3@vu.nl

Weikang Weng
Leiden University
Leiden, Netherlands
w.weng@liacs.leidenuniv.nl

Vincent François-Lavet
VU Amsterdam
Amsterdam, Netherlands
vincent.francoislavet@vu.nl

Rob van Nieuwpoort
Leiden University
Leiden, Netherlands
r.v.van.nieuwpoort@liacs.leidenuniv.nl

Alexandru Uta
DFINITY
Zurich, Switzerland
alexandru.uta@gmail.com

ABSTRACT

Serverless cloud platforms typically rely on operating-system schedulers such as the Completely Fair Scheduler (CFS) and Earliest Eligible Virtual Deadline First (EEVDF) to schedule function executions across CPU cores. Prior work has shown that existing schedulers either minimize task latency at the expense of runtime efficiency or improve runtime efficiency while increasing latency. Although hybrid approaches partially mitigate this trade-off, each scheduler remains confined to a narrow region of the latency–runtime trade-off space, leaving much of the design space unexplored.

We present a class of synthesized schedulers that expose configurable latency–runtime trade-offs. Rather than hand-designing scheduling heuristics, we use reinforcement learning to synthesize scheduling policies for core assignment, preemption, and load-balancing optimized for different reward objectives. We implement the synthesized schedulers using Linux `sched_ext`, train them on representative serverless workloads, and evaluate them on a multi-core server. Our evaluation shows that synthesized schedulers are competitive with both general-purpose Linux schedulers (CFS, EEVDF, and FIFO) and state-of-the-art serverless-aware schedulers. More importantly, synthesized schedulers can be tuned at deployment time to explore a near-continuous set of operating points spanning the latency–runtime trade-off. We show that several synthesized operating points reduce both latency and runtime cost, improving profitability for both cloud users and providers.

1 INTRODUCTION

Serverless computing has emerged as a cloud paradigm in which users run functions without managing the servers themselves. Most serverless platforms are built on Function-as-a-Service (FaaS), which executes functions in response to events and automatically scales resources with workload demand [21, 27, 59, 62, 66, 74]. This abstraction simplifies application deployment and shifts the responsibility for efficient execution to the cloud provider. Because users are charged under a pay-as-you-go model, they only pay for the runtime of function invocations [4, 12, 13, 27]. Once functions are running on a worker node, their latency, runtime, and billing cost are strongly affected by how they are scheduled on the server. Thus, node-level scheduling decisions directly influence both user-perceived performance and provider-side efficiency.

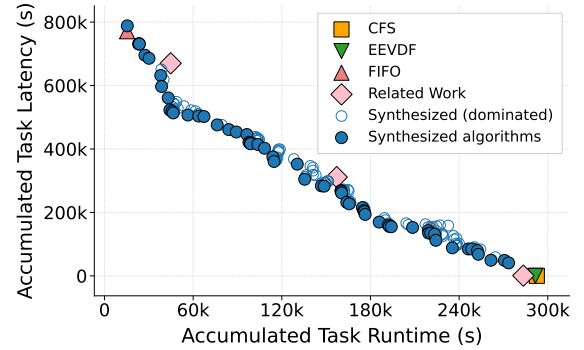


Figure 1: Latency–runtime trade-off. Accumulated task latency and accumulated task runtime across a workload extracted from Microsoft Azure trace. CFS and EEVDF achieve low latency with high runtime, while FIFO reduces runtime at the cost of higher latency. Our synthesized schedulers trace out an empirical Pareto frontier.

OS-level scheduling practice has not kept pace with this shift: node-level scheduling still relies on general-purpose OS schedulers that were not designed for serverless execution [15, 16, 75], and they can become a performance bottleneck. Traditional Linux schedulers, such as the Completely Fair Scheduler (CFS) [8] and the Earliest Eligible Virtual Deadline First Scheduler (EEVDF) [11], aim to balance fairness, latency, and throughput across diverse workloads [34, 43]. However, serverless workloads differ from the long-running processes for which these schedulers are typically optimized: they are characterized by high concurrency, bursty invocations, and short-lived tasks that often complete in milliseconds to seconds [28, 50, 62].

Under such conditions, fairness mechanisms across tasks based on historical execution can be less effective, because many functions finish before the scheduler can accumulate meaningful execution history [34]. Moreover, serverless workers are often highly over-committed, causing frequent context switches, preemptions, and contention for shared hardware resources such as CPU cores, memory bandwidth, and last-level cache [18, 22, 48, 63]. These effects can increase the task runtime and therefore billing cost [15, 22, 40, 75], while also degrading user-perceived latency [35, 65]. To address these latency and contention problems, prior low-latency scheduling systems introduce mechanisms such as centralized dispatch, preemption, and core-granular load balancing [23, 29, 30]. These

policies, however, are fixed: each offers a single scheduling behavior, whereas serverless applications are highly heterogeneous—spanning latency-critical to cost-sensitive requirements [28, 50, 62] and shifting across execution phases—that no single predetermined policy can satisfy at once.

These limitations motivate reasoning about the full space of scheduling behaviors rather than any single fixed policy. In practice, cloud servers handling serverless workloads are provisioned for long periods of time, continuously running functions for spans such as weeks, months, or longer. To increase profits, providers keep these machines continuously at a high utilization level [3, 5], making the OS scheduler one of the central parts of the cloud stack. When servers are continuously running tasks, there are two important high-level metrics that can be used to reason about performance: function latency and function runtime. Both are user-facing, with the former having a high impact in deadline-constrained tasks, while the latter is directly proportional to billed cost [4]. Moreover, runtime is extremely important for providers as well: optimizing for short runtimes leaves room for improved overall throughput and cost consolidation. Therefore, when considering a period of time in the life of a cloud server, *accumulated latency* and *accumulated runtime* paint a clear picture of the OS scheduler performance, e.g., as presented in Figure 1. Existing policies such as CFS, EEVDF or FIFO occupy distinct operating points within the broader latency–runtime trade-off space.

This raises the central question of this paper: what operating points are available in the latency–runtime trade-off space for serverless OS scheduling, and can OS schedulers be systematically tuned to move toward different objectives? Instead of committing to a single fixed objective, we explore an empirical Pareto frontier between accumulated task latency and accumulated task runtime. This perspective is important because CFS, EEVDF, and FIFO occupy different regions of the trade-off space: CFS/EEVDF reduce waiting time but incur longer runtime (hence higher user cost), while FIFO preserves runtime efficiency but increases user-perceived latency.

Learning-based scheduling provides a way to synthesize policies for different points in this latency–runtime trade-off space. Reinforcement learning (RL) is a good fit for this setting because scheduling is a sequential decision-making problem in which actions can be evaluated through feedback from the environment [46, 64]. Recent studies have applied RL to serverless environments, mainly focusing on fleet optimization and resource management [57, 71]. However, these approaches treat each worker node as a black box. After a function is assigned to a server, its execution is governed by the local OS scheduler. As a result, cluster-level placement alone cannot address local effects such as context switching, queue imbalances, and preemption behavior. These effects directly influence function latency, runtime, and therefore billing cost.

To explore the empirical latency–runtime Pareto space (Figure 1), we present an architecture for OS scheduler synthesis. It uses RL algorithms to drive low-level OS scheduling mechanisms under different reward objectives. Applying RL at the OS scheduling layer, however, requires decisions at much finer timescales than typical learning-based resource managers can support. We therefore separate scheduling control across two timescales, amortizing inference through batched task placement while adapting system-level

preemption and rebalancing less frequently. Our synthesized schedulers use `sched_ext` [60] to achieve efficient user-space scheduling. Schedulers use two RL policies: the high-frequency policy handles CPU core assignment for arriving serverless tasks, while the low-frequency policy observes global system state, such as queue lengths, to adjust the preemption time slice and to perform load balancing. By observing the runtime state of the execution environment, the two policies learn to optimize end-to-end objectives such as accumulated task latency and accumulated task runtime. In this article, we make the following contributions:

- We design and implement an architecture to synthesize OS-level schedulers for serverless functions. Based on RL-driven decisions, the synthesized schedulers couple high-frequency task-dispatch decisions with a lower-frequency controller that adjusts preemption and load rebalancing.
- We synthesize and deploy schedulers on multi-core systems using a modern extensible scheduling framework (`sched_ext`) and evaluate them with real-world FaaS workload traces. Our synthesized schedulers are competitive with both general-purpose (CFS, EEVDF, FIFO) and serverless-aware (Hybrid [75], ALPS [16], SFS [15]) schedulers.
- We explore and characterize the empirical latency–runtime Pareto space of serverless OS scheduling. We show that synthesized schedulers offer trade-off points beyond what existing schedulers can offer, leading to increased profitability for both clients and cloud providers.

2 BACKGROUND

In this section, we introduce background knowledge for OS-Level scheduling for serverless, reinforcement learning, and two reinforcement learning methods used in this article.

2.1 OS-Level Scheduling for Serverless

A serverless worker executes many concurrent function invocations on a multi-core machine. Once functions are assigned to the worker, their latency and runtime are determined not only by the function itself but also by the OS-level scheduler. In this work, we define task latency as the elapsed time from arrival to first execution, and task runtime as the elapsed time from first execution to its completion. Under this definition, runtime can increase when a task is preempted, requeued, or migrated after it has started. The scheduler therefore affects both when a function first executes and how long it takes to complete after starting.

The system must make online decisions under limited information. In particular, task runtimes are unknown a priori and may vary significantly across invocations, preventing accurate workload prediction. The scheduler observes only coarse-grained system state, such as per-core queue lengths/occupancy, rather than detailed per-task remaining processing time. Arriving tasks are independent and must be assigned without knowledge of future arrivals. Additionally, once dispatched, tasks may be subject to system constraints such as non-negligible context switching or migration overheads. These factors jointly make the scheduling problem challenging, as decisions must balance load distribution and latency with only partial information about the workload or future arrivals.

A scheduler decides where arriving functions are queued, when running functions should be preempted, and whether queued work should be rebalanced across cores. These decisions create a latency–runtime trade-off. Giving arriving functions earlier CPU access can reduce latency, but frequent preemption may increase runtime because a preempted task’s measured runtime continues to grow while it waits to resume execution, in addition to context-switch overhead and reduced cache locality. In contrast, run-to-completion execution can preserve runtime efficiency, but may cause head-of-line blocking when long functions occupy per-core queues. Similarly, moving tasks across cores can reduce queue imbalance, but excessive migration may introduce overhead. These mechanisms form the core scheduling space explored in this work.

2.2 Reinforcement Learning

Reinforcement learning (RL) [14, 64] addresses sequential decision-making problems that are typically formalized as a Markov Decision Process (MDP). An MDP is defined by a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s'|s, a)$ is the state transition probability, $R(s, a)$ is the reward function, and $\gamma \in [0, 1)$ is the discount factor. At each time step t , an agent observes state $s_t \in \mathcal{S}$, selects action $a_t \in \mathcal{A}$ according to a policy $\pi(a_t|s_t)$, receives reward $r_t = R(s_t, a_t)$, and transitions to the next state $s_{t+1} \sim P(\cdot|s_t, a_t)$. The goal of RL is to find an optimal policy π^* that maximizes the expected cumulative discounted reward:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right] \quad (1)$$

One approach to learning the optimal policy is through value-based methods, which first learn a state-action value function (Q-function) $Q^{\pi}(s, a)$ that represents the expected return when taking action a in state s and following policy π thereafter:

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s, a_t = a \right] \quad (2)$$

Once the optimal Q-function Q^* is learned, the optimal policy can be derived by acting greedily with respect to the Q-values: $\pi^*(s) = \arg \max_a Q^*(s, a)$.

2.3 Deep Q-Networks (DQN)

Deep Q-Networks (DQN) [49] integrate deep neural networks into this process to estimate action values, allowing RL to scale up to complex, high-dimensional environments. The network is trained using temporal difference learning, which updates the internal weights by minimizing the difference between the current value prediction and an updated target value. This target value combines the immediate reward received with the discounted maximum predicted value of the next state.

To prevent the training process from becoming unstable, the architecture introduces two key mechanisms. **Experience Replay:** The agent stores its historical experiences into a buffer and then the network trains on small groups of these experiences sampled completely at random, which breaks the problematic chronological correlation between consecutive steps. **Target Network:** A separate copy of the neural network is used exclusively to generate the target values for training. Its internal weights are kept frozen

and only updated to match the active network at fixed intervals, preventing the target values from shifting too rapidly and confusing the optimization process.

2.4 Dueling DQN

Standard single network architectures (like DQN) do not explicitly separate a state’s baseline value from the relative advantage of each action, but instead learn a single Q-value per action directly. This can be wasteful when actions yield similarly valued outcomes, such as a navigation task where an agent far from any obstacle gains little from distinguishing between directions.

The Dueling architecture [68] addresses this inefficiency by splitting the internal structure of the network into two distinct streams: a state-value function that estimates the overall benefit of the current state, and an advantage function that determines the relative merit of each specific action compared to the others. Both streams share a primary feature-extracting layer but branch into separate heads before merging to output the final action values. To ensure that the system can uniquely identify the individual contributions of the state value and the action advantages, the advantage stream is centered by subtracting its average across all possible choices. The entire framework is trained end-to-end using the same error-minimization process as the standard approach.

3 SYSTEM MODEL

In this section, we explain the main design choices behind our synthesized scheduler before formalizing the scheduling problem. We start from a simple scheduler that assigns functions to per-core FIFO queues and show why two scheduling mechanisms, preemption and work shuffling, are needed for serverless workloads (Section 3.1). We then justify the choice of Q-learning as the RL algorithm for our synthesized scheduler (Section 3.3). Next, we explain why a global queue is necessary to expose system-wide state and support batched decisions in our reinforcement learning architecture (Section 3.2). Finally, we define the scheduling problem and its state/action spaces (Section 3.4).

3.1 Scheduling Mechanisms

In serverless clouds, the underlying servers run for weeks or months and are kept busy at high utilization [3, 5]. The OS scheduler decides how the incoming functions are placed and run on servers, hence the scheduler has a direct effect on two metrics that both users and cloud providers deem important. One is latency, which has impact on deadline-constrained functions. The other is runtime, which decides how much the user is billed under a pay-as-you-go pricing model [4]. For the provider, runtime also influences how much capacity there is to improve the overall throughput and achieve cost consolidation. We therefore measure a scheduler by its *accumulated latency* and *accumulated runtime* over a workload, and use these two metrics to motivate the system model below.

When building our synthesized schedulers, we build on mechanisms used by existing schedulers: global/per-core queues, preemption, and work shuffling. However, we use RL policies to steer these mechanisms. To showcase the integration of preemption and work shuffling into our synthesized scheduler, we first conduct a baseline experiment. We evaluate a workload of 9,463 serverless

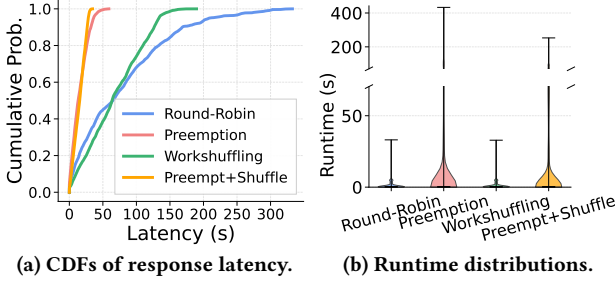


Figure 2: CDFs of response latency and runtime distributions with a workload under Round Robin, Preemption, Work shuffling on 50 CPU cores.

functions on 50 CPU cores and compare the latency distribution and runtime distribution under four scheduling mechanisms, which are as follows:

Round-Robin. In *round-robin*, tasks are assigned to per-core queues in a round-robin manner and are executed in a First-In-First-Out (FIFO) order. Neither preemption nor work shuffling is enabled. This scheduler serves as our baseline.

Preemption. Tasks are initially assigned in a round-robin manner and executed in FIFO order. A fixed time slice of 1,000 ms is applied. Any task exceeding this runtime quota is preempted and requeued at the end of its per-core queue.

Work Shuffling is a load balance scheduling mechanism. Tasks are initially assigned in a round-robin manner and executed in FIFO order. When the most heavily loaded core has more than 20 queued tasks, the scheduler migrates half of the queue-length difference to the least-loaded core. To avoid migration thrashing (i.e., the ping-pong effect), shuffling is triggered only when the queue length difference exceeds 25% of the maximum queue length.

Figures 2a and 2b show the latency and runtime distributions, respectively. As shown, *Preemption* mechanism reduces latency for shorter functions by mitigating head-of-line blocking. However, this benefit comes at the cost of longer runtime, defined as the duration from first time run to final completion. This is because preempted tasks may incur additional context-switch overhead and waiting time after being requeued. By contrast, *Work Shuffling* targets a different bottleneck. Rather than interrupting running tasks, it mitigates spatial queue imbalance across CPU cores. As shown in Table 1, work shuffling mechanism reduces long-tail effect compared to other scheduling mechanisms. This improvement occurs because the workload is less likely to be bottlenecked by a few congested per-core queues, thereby reducing tail delays that would otherwise prolong the entire workload’s completion time.

In summary, *Preemption* and *Work Shuffling* improve different aspects of serverless scheduling performance. *Preemption* reduces short-function latency by mitigating head-of-line blocking, but it may increase runtime due to requeueing and context-switch overhead. *Work Shuffling* reduces the long-tail effect by mitigating spatial queue imbalance across CPU cores.

3.2 Scheduler Global Queue

We motivate the use of a global queue in our reinforcement learning architecture. The global queue provides two key benefits.

1. Global State Visibility. Per-core queues only expose local load information. In contrast, a global queue allows the RL agent

Table 1: Comparison of the time required to complete 99% of the tasks under four scheduling mechanisms.

Mechanisms	Time to Complete 99% of Tasks (s)
Round-Robin	388
Preemption	294
Work Shuffling	268
Preempt+Shuffle	254

to observe the system-wide arrival pattern and the distribution of pending work across CPU core queues. This visibility is important for making spatial load-balancing decisions and for penalizing queue imbalance during learning.

2. Temporal Batching and Burst Awareness. Serverless workloads are often bursty, and dispatching each task immediately upon arrival would force the scheduler to make decisions at a very high frequency. Instead, the global queue buffers incoming tasks over short time windows, such as 20 ms, and dispatches them as a batch. This reduces the frequency of RL inference and allows the scheduler to estimate the arrival intensity of incoming traffic. For example, if a batch contains 100 tasks, the scheduler can infer that the system is experiencing a burst and may apply more aggressive load balancing or preemption. Conversely, if a batch contains only a few tasks, the scheduler may choose a more conservative policy.

3.3 Q-learning

The observation above suggests that *Preemption* and *Work Shuffling* are not universally beneficial. Their effectiveness depends on the dynamic system state, including workload burstiness, queue imbalances, and workload characteristics. This motivates an adaptive scheduling policy that can learn when these mechanisms are beneficial and can apply them selectively. Such a policy is necessary because a decision that improves latency for one function may increase its runtime.

This setting is naturally sequential: each scheduling decision changes the subsequent queueing state. Moreover, future arrivals are uncertain and a limited set of features describes the system, making explicit planning impractical. We therefore formulate this adaptive scheduling problem as a reinforcement learning problem, in which the scheduler observes the current system state and selects scheduling actions to optimize overall performance. Concretely, the scheduler selects a destination CPU core for each incoming function, chooses a preemption time slice from a finite set, and selects destinations for tasks to be shuffled. All of these actions are discrete and the action space is small, which makes value-based methods a natural and sample-efficient fit. To handle the high-dimensional state space and exploit the generalization of neural networks, we use deep Q-learning, specifically DQN [49] and Dueling DQN [68]. Policy-gradient methods such as DDPG [39], TD3 [17], and SAC [20], by contrast, target continuous action spaces and would require modification to fit our discrete formulation. Discrete policy-gradient methods are also applicable to our setting, and we leave a comparison with them for future work.

In practice, a scheduling decision spans core assignment, the preemption time slice, and load rebalancing. Rather than encoding all of these into a single large discrete action space, we decompose control into two value-based agents: a high-frequency agent that selects destination cores for spatial placement, and a low-frequency

Table 2: State and action spaces of the RL agent (in parentheses, the number of discrete choices for each action).

State Space
Number of started tasks, number of unstarted tasks, number of finished tasks, per-core queue lengths, accumulated latency, accumulated runtime, accumulated preemption count
Action Space
Destination CPU core for incoming invocations (50 choices DDQN [68]), preemption time slice (5 choices, DQN [49]), destination CPU cores for rebalanced tasks (50 choices, DDQN [68])

agent that selects the preemption time slice for temporal control. Load rebalancing reuses the placement agent to choose migration targets. Because each agent operates over a much smaller action space than their combination, the two spaces stay compact, which in turn stabilizes learning and improves sample efficiency.

3.4 Objectives

We model serverless function scheduling as an MDP. The RL agent observes the current execution state and incoming function invocations, then selects scheduling actions, including the target CPU core, preemption time slice, and shuffle destination. The resulting state and action spaces are summarized in Table 2.

Since serverless platforms continuously execute function invocations, scheduling performance can be characterized by two key metrics: latency and runtime. Latency is critical for deadline-constrained user-facing workloads, whereas runtime is directly related to billed cost under the pay-as-you-go pricing model. The objective of the RL agent is to maximize cumulative reward over workload execution while optimizing *accumulated task latency* and *accumulated task runtime*. We aim to synthesize two classes of schedulers, each optimized for a different reward objective:

Latency-oriented schedulers optimize accumulated task latency, defined as the sum of the time from function arrival to first execution across all tasks. This objective is important for user-facing services, where low latency directly affects the user experience. Prior work reports that a one-second slowdown can reduce customer satisfaction by 16% [25, 33].

Runtime-oriented schedulers optimize accumulated task runtime, defined as the sum of the time from first execution to completion across all tasks. This objective is important because users are charged under a pay-as-you-go pricing model, where they pay for the runtime of their functions [12, 13, 27].

4 SYNTHESIZING SCHEDULERS

This section presents the design and implementation of our scheduler-synthesis architecture. We first give an overview of the architecture, then describe the reward designs that steer a policy toward low latency or low runtime. We then show how we train convergent policies, how a trained policy is tuned to explore the latency-runtime trade-off, and how the scheduler is implemented on sched_ext.

4.1 Architecture Overview

Driven by the motivation in Section 3, we propose a scheduler-synthesis architecture based on reinforcement learning.

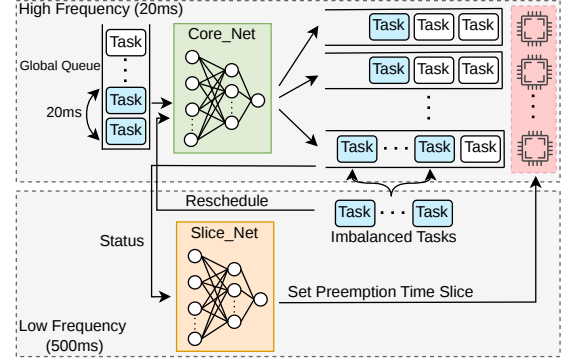


Figure 3: Architecture of synthesized schedulers. The high-frequency controller batches arriving tasks and assigns them to per-core queues every 20 ms. The low-frequency controller adjusts the preemption time slice and performs load rebalancing every 500 ms.

System Overview. Our schedulers follow a centralized, single-dispatcher model. The kernel-side BPF backend enqueues every runnable task into a global shared queue (a ring buffer spanning all CPUs), which serves as the system-wide global queue. Tasks arriving at the system are initially buffered in this global queue. A single user-space scheduler thread drains the global queue, decides placement, and dispatches each task to the tail of a specific per-core queue; each worker core owns one such per-core queue, within which tasks are processed in a FIFO manner. To prevent head-of-line blocking, a time-slice preemption mechanism is applied. If a task runs out of its allocated execution quota before completion, it is immediately preempted and requeued at its per-core queue’s tail.

The synthesized scheduler decides both where a task runs (its spatial placement) and when it runs (its timing), so the two are coordinated rather than handled separately. The scheduler contains two learned policies that operate at different frequencies: the high-frequency policy π_{HF} , realized by Core_Net (a Dueling DQN), and the low-frequency policy π_{LF} , realized by Slice_Net (a DQN). The high-frequency policy handles spatial placement every 20 ms: at each dispatch epoch, it collects the tasks accumulated in the global queue and queries π_{HF} to assign each invocation to a worker CPU core. The low-frequency policy performs system-level tuning and rebalancing every 500 ms: it queries π_{LF} to adapt the global preemption time slice applied to tasks. In the same control step, it also evaluates the per-core load distribution and, when severe imbalance is detected, triggers work shuffling by migrating queued tasks from congested cores to less loaded ones. The complete architecture of our synthesized schedulers is illustrated in Figure 3.

High-Frequency Policy (Spatial placement). Operating at a fine-grained interval (e.g., 20 ms), the high-frequency policy acts as the system’s spatial task dispatcher. Instead of scheduling tasks individually upon arrival, it accumulates a batch of incoming tasks within this short window and performs a single batched inference. The policy π_{HF} observes the current system state s (e.g., per-core queue lengths, accumulated preemption count) and the number of batched tasks n , and outputs a vector of destination cores $\mathbf{c} = \pi_{HF}(s, n)$, where $\mathbf{c} = (c_1, \dots, c_n)$ and c_i is the worker core selected

for the i -th task. The scheduler then dispatches each task to its per-core queue. By fixing the batching interval but letting the batch size n vary with the arrival rate, the agent observes bursty workloads' instantaneous density and learns adaptive placement.

Low-Frequency Policy (Temporal Tuning & Rebalancing).

Operating at a coarser interval (e.g., 500 ms), the low-frequency policy performs system-level temporal adjustment and spatial rebalancing. The policy π_{LF} observes the global system state and selects a preemption time slice. On the worker cores, tasks are bound by this slice: once a task exhausts its quota, it is preempted. At the same interval, the low-frequency policy also evaluates the load distribution and triggers RL-driven work shuffling, migrating tasks from congested queues to idle ones to resolve spatial imbalance.

The **RL Load Rebalancing Mechanism** first identifies the globally least-loaded worker CPU core with L_{min} tasks and sorts all worker CPU cores in descending order based on their individual queue lengths. To detect load imbalance, the mechanism evaluates each CPU core in the sorted list against the minimum load using two conservative threshold-based conditions: it triggers migration if $L_{min} = 0$ and $L \geq \theta_{idle}$, or if $L > \theta_{load}$ and $L - L_{min} \geq \theta_{gap}$, where L is the queue length of the currently evaluated CPU core. In our implementation, we set $\theta_{idle} = 2$, $\theta_{load} = 10$, and $\theta_{gap} = 5$ to avoid triggering migration for minor queue fluctuations and to rebalance only when idle capacity or substantial localized congestion exists. For each qualifying CPU core, the mechanism determines the number of tasks to migrate, defined as half the difference between L and L_{min} , and dequeues these tasks from the back of its queue. Finally, the load balancer executes a single batched inference request for all collected tasks through the Core_Net in the Python-side RL agent, and distributes them to destination worker CPU cores based on the sequential decisions returned by the RL agent.

Core_Net and Slice_Net are the two value networks used by the synthesized scheduler. Core_Net selects destination CPU cores for tasks accumulated in the global queue. This decision has a relatively large discrete action space, since the agent chooses among worker cores. We therefore use a Dueling Deep Q-Network (Dueling DQN) [68] for Core_Net, which can improve learning when many actions have similar values. Slice_Net selects the preemption time slice for worker tasks. Its action space is much smaller, consisting of five candidate time slices. Four of these candidates, 32 ms, 174 ms, 390 ms, and 1,633 ms, correspond to the 25th, 50th, 75th, and 90th percentiles of our sampled workload durations and are shared by both objectives. The two objectives differ only in the fifth, extreme candidate. For the latency-oriented scheduler we add an aggressive 8 ms slice, the 10th percentile of workload durations. A short slice forces running tasks to give up the CPU quickly, so a newly arrived task waits less before its first run, which is exactly what the latency objective rewards. For the runtime-oriented scheduler we instead replace this aggressive option with non-preemptive run-to-completion. Here the trade-off runs the opposite way: short slices keep interrupting tasks and switching them between cores, so each task takes longer to finish. Running a task to completion avoids these interruptions and keeps its data on the same core, which in turn lowers the total runtime. Since this is a small discrete action space, we use a simpler Deep Q-Network (DQN) [49] for Slice_Net to reduce model complexity.

State Space and Normalization. Both networks observe a shared RL state consisting of the number of started, unstarted, and finished tasks, together with the accumulated latency, accumulated runtime, and accumulated preemption count. Because these quantities feed the network as inputs (distinct from the reward terms defined below), we normalize them to keep training stable: the three task counts are passed through a logarithmic transformation $f(x) = \log(1 + x)$ to compress their dynamic range and dampen the effect of bursty arrivals, while the temporal and event-based metrics (accumulated latency, accumulated runtime, and preemption count) are linearly rescaled by fixed denominators. This maps the inputs into a bounded numerical range and prevents vanishing and exploding gradients caused by high-magnitude raw counts.

4.2 Optimizing Accumulated Task Latency

According to Little's Law ($L = \lambda W$) [41], the long-term average number of tasks waiting in a system (L) is strictly proportional to the average waiting time (W) given a stable arrival rate (λ). Inspired by this principle, minimizing the accumulation of waiting tasks in the queue intuitively minimizes the expected latency for each task. Formally, the total accumulated latency can be formulated as the sum of the waiting times for all tasks:

$$T_{\text{accumulated task latency}} = \sum_i (\text{first execution}_i - \text{arrival}_i) \quad (3)$$

Let $N_{\text{unstarted}}(t)$ denote the number of tasks that have arrived but have not yet started executing at time t . The optimization objective is equivalent to minimizing the time integral of all waiting tasks over the scheduling period:

$$\min \sum_i (\text{first execution}_i - \text{arrival}_i) = \int_0^T N_{\text{unstarted}}(t) dt \quad (4)$$

This single count, however, hides where tasks are waiting. If some cores have far longer queues than others, tasks on those cores must wait behind many earlier tasks (head-of-line blocking) even while other cores have nothing to run. To capture this queue imbalance, we define a single *imbalance* penalty $I(t)$ from the per-core queue lengths $Q_k(t)$ (with mean $\bar{Q}$, standard deviation σ_Q , and maximum $Q_{\max}$), which is reused by both reward functions:

$$I(t) = w_\sigma \frac{\sigma_Q(t)}{\max(1, \bar{Q}(t))} + w_b \frac{Q_{\max}(t)}{\max(1, \bar{Q}(t))}. \quad (5)$$

The first term is the coefficient of variation of the queue lengths and the second is how much the busiest core exceeds the average; w_σ and w_b are objective-specific weights fixed within each experiment. We floor the denominator at one task so that $I(t)$ is well-defined and equals zero when the system is idle (all per-core queues empty). Symmetrically, we define a *waiting-work* penalty $W(t)$ that measures how much work is waiting, from the number of unstarted tasks together with the mean and maximum queue lengths:

$$W(t) = w_1 \log(1 + N_{\text{unstarted}}) + w_2 \log(1 + \bar{Q}) + w_3 \log(1 + Q_{\max}). \quad (6)$$

All counts are log-scaled so that no single term dominates when queues grow to hundreds of tasks during a burst. The system *pressure* $P(t)$ is then simply the sum of the two penalties:

$$P(t) = W(t) + I(t). \quad (7)$$

The number of unstarted tasks leads with the largest weight w_1 because, by Little’s Law, it is the objective; the queue-length and imbalance terms are secondary signals that discourage the imbalanced states causing head-of-line blocking and starvation. Across all latency-oriented experiments we fix $(w_1, w_2, w_3) = (2.0, 0.5, 1.0)$ and $(w_\sigma, w_b) = (0.5, 0.3)$.

Finally, we convert this pressure into a per-step difference-based reward. Let $\Phi(t) = -\log(1 + P(t))$, where lower pressure corresponds to a larger potential value. We define the reward as the change in this potential between consecutive scheduling steps:

$$r_t = \Phi(t+1) - \Phi(t). \quad (8)$$

Each step therefore rewards the agent for reducing pressure. Under the discounted return optimized by DQN, this difference-based reward favors policies that reduce pressure earlier and keep it low throughout the episode.

4.3 Optimizing Accumulated Task Runtime

The runtime of a task is defined as the duration from its first execution on CPU to its final completion. The accumulated runtime across all tasks is formulated as:

$$T_{\text{accumulated task runtime}} = \sum_i (\text{completion}_i - \text{first execution}_i) \quad (9)$$

In our setting, the task runtime can be prolonged by scheduling-induced overheads, such as delays caused by preemption and re-queuing. A highly imbalanced queue distribution can cause idle cores to steal tasks from busy cores, leading to frequent preemptions and prolonged runtime. Since the final task runtime is only observed at completion, we use runtime-related signals observable during scheduling: the preemption rate and queue imbalance.

The dominant cause is preemption, so here the primary term is the preemption rate: the number of preemptions $\Delta P(t)$ per scheduling step normalized by the core count K :

$$\rho_P(t) = \frac{\Delta P(t)}{K}. \quad (10)$$

We normalize by the number of cores rather than by the number of queued tasks so that a step full of preemptions stays clearly distinguishable from a preemption-free step even when the queues are deep. Reusing the same imbalance penalty $I(t)$ from Eq. (5) (since a lopsided queue lets idle cores steal tasks from busy ones and triggers extra preemptions), the runtime pressure has the same *primary + imbalance* form as before:

$$P_{\text{exec}}(t) = w_p \rho_P(t) + I(t). \quad (11)$$

The preemption rate carries by far the largest weight because it is the most direct driver of runtime; the imbalance penalty plays the same secondary, locality-preserving role as in the latency reward, with smaller weights. Across all runtime-oriented experiments we fix $w_p = 4.0$ and $(w_\sigma, w_b) = (0.2, 0.05)$.

We map this pressure to a bounded per-step reward with a logarithm:

$$r_t = -\log(1 + P_{\text{exec}}(t)), \quad (12)$$

which keeps rewards in a stable range and yields diminishing penalties as pressure grows. Maximizing $\sum_t r_t$ trains a policy that avoids needless preemption and severe queue imbalance, preserving locality and shortening accumulated task runtime.

4.4 Training Convergent Models

We train each scheduler on the training workload for 50 episodes using five independent random seeds. Figure 4 shows the reward and the corresponding runtime metrics with 95% confidence intervals over the 50 training episodes. The results show that both reward and latency/runtime stabilize within 50 episodes, indicating that the synthesized scheduler can learn convergent policies for core assignment, preemption, and load rebalancing.

For each episode, we compute the mean and confidence interval across the five runs with the same episode index. The standard error of the mean (SEM) is computed as $\text{SEM} = \frac{\sigma}{\sqrt{n}}$, where σ is the sample standard deviation across runs and $n = 5$ is the number of runs. We use a normal-approximation 95% confidence interval, computed as mean $\pm 1.96 \times \text{SEM}$. For visualization, we apply a 5-episode moving average to smooth both mean curves and confidence bands.

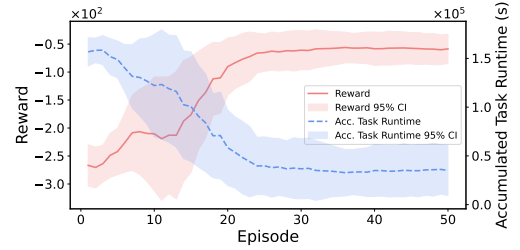


Figure 4: Reward and accumulated runtime over 50 training episodes, with 95% confidence bands across five random seeds. Both stabilize within 50 episodes, indicating convergence.

4.5 Exploring Latency–Runtime Pareto Space

In this section, we show that a single trained policy can be tuned to trace out the latency–runtime trade-off, without retraining a separate model for each operating point. We take the runtime-optimized Dueling DQN policy and, at evaluation time, hold its placement behavior fixed while tuning two parameters that control preemption. The first is the preemption time slice: a short slice interrupts running tasks quickly, so newly arrived tasks reach the CPU sooner (lower accumulated latency) at the price of more preemptions and context switches (higher accumulated runtime), whereas a long slice lets tasks run largely to completion (lower runtime) at the price of longer waiting (higher latency). The slice alone, however, moves the operating point in discrete jumps: because the workload’s execution times are clustered, shrinking the slice pushes whole groups of tasks past the preemption threshold at once, leaving gaps in the trade-off. To fill these gaps, we tune a second parameter: a global budget on the total number of task preemptions per episode. Varying this budget changes the total preemption count, and hence accumulated latency and runtime. Tuning both parameters yields a near-continuous set of operating points spanning the latency–runtime trade-off, as seen in Figure 1.

A natural question is whether a heuristic scheduler such as CFS can reach this near-continuous set of operating points simply by tuning the preemption parameters. In our exploration, we hold the learned placement policy fixed and vary only preemption. Therefore, the set of operating points is decided not only by the preemption parameters but also by the placement policy. CFS places and

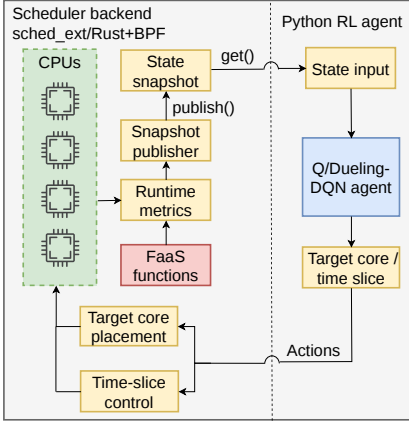


Figure 5: Synthesized scheduler communication pattern: the sched_ext Rust+BPF backend (left) publishes a state snapshot of runtime metrics to the Python RL agent (right); the Q/Dueling-DQN agent returns target-core and time-slice decisions, which the backend applies as core placement and time-slice control.

migrates tasks with a fairness-driven, serverless-agnostic load balancer, which prolongs runtime through preemptions. Thus, tuning its time slice still traces a set of operating points in the high-runtime region. Furthermore, CFS also exposes no budget on the total number of preemptions. Its preemptions follow fairness among tasks instead of a tunable global budget, leaving it unable to fill the gaps left by the slice alone. Thus, tuning CFS yields a set of operating points in the high-runtime region instead of the near-continuous set our synthesized schedulers can reach.

This limitation is not specific to CFS. Each heuristic scheduler that we evaluate is designed around a single objective and therefore occupies one region of the trade-off: ALPS [16], CFS, and EEVDF preempt aggressively and occupy the high-runtime region; FIFO and SFS [15] preempt much less and occupy the high-latency region; Hybrid [75] occupies an intermediate region. These schedulers expose tunable parameters, but none of them decouples placement from preemption the way synthesized schedulers do. Our synthesized schedulers decouple placement from preemption: they keep a learned placement policy fixed while independently tuning the preemption slice and a global preemption budget. This separation allows deployment-time tuning to expose a near-continuous set of operating points without retraining the scheduler.

4.6 Implementation

We implement synthesized schedulers on top of sched_ext [60], a modern framework for implementing custom Linux schedulers with BPF. Our implementation follows a high-level architecture: a scheduler backend, a Python-based RL agent, and a synchronization bridge between the scheduler and the learning logic. Figure 5 shows the interaction of these components.

sched_ext backend. Synthesized schedulers use a Rust user-space component together with a BPF scheduling component. The BPF component is the kernel-side interface to sched_ext: it receives scheduling callbacks, forwards runnable-task events to the

Rust scheduler, and applies scheduling decisions by dispatching tasks into sched_ext dispatch queues (DSQs). A DSQ is the kernel-side dispatch queue where CPU cores pick runnable tasks from. The Rust user-space maintains a global queue and per-core queues, invokes the RL agent to make placement and rebalancing decisions, and returns target cores and time slices to the kernel interface.

RL agent. The RL agent is implemented in Python using PyTorch [53], and produces both CPU-placement decisions and time-slice decisions using Deep Q-learning and Dueling Deep Q-Networks. Python communicates with the Rust scheduler using *PyO3* [54]. The scheduler exposes state snapshots to Python, including queue lengths, numbers of started, unstarted, and finished tasks, latency and runtime statistics, and preemption information. The Python agent uses these snapshots to compute rewards and update the Q-network.

Synchronization. To keep Python inference off the scheduler fast path, the RL agent runs on a dedicated core outside the worker set, so its inference and training never compete with scheduled tasks. Each round, the Rust scheduler publishes the latest state snapshot into a shared in-process slot that the embedded Python interpreter reads, and then invokes the agent through *PyO3* to obtain placement and time-slice actions. Decisions run at two frequencies to limit how often Python is called. Placement is updated every 20 ms, while the time slice and load rebalancing are updated every 500 ms.

5 EXPERIMENTAL SETUP

In this section, we first describe the hardware and software setup (Section 5.1). We then introduce the workloads and metrics used in our experiments (Section 5.2).

5.1 Hardware and Software Setup

We use a server provided by CloudLab [10] with two Intel Xeon Gold 6142 CPUs (2.60 GHz, 16 cores each with hyper-threading) providing 64 CPU cores in total, and 384 GB of DRAM. We dedicate 50 of these cores to the sched_ext worker pool; this leaves headroom for the user-space scheduler, the RL agent, and background system activity, so that the agent’s PyTorch inference and training do not compete with scheduled tasks. All main experiments run the scheduler policy on CPUs, with CUDA disabled in PyTorch. GPUs are used only in a separate microbenchmark that compares the same PyTorch policy-inference path with CUDA disabled and enabled on an NVIDIA A30. **OS:** Ubuntu 26.04 LTS (Linux 7.0.0-10-generic) with sched_ext enabled. For a fair comparison, we reimplemented all evaluated schedulers in sched_ext, following their original designs.

5.2 Workloads and Metrics

We train on serverless function arrival patterns extracted from the real-world Microsoft Azure FaaS trace [62], following the trace-extraction methodology of prior work [15, 16, 75]. The training workload is the arrival sequence from the first two minutes of Day 1, whose execution durations are highly heterogeneous (8–25,280 ms), preserving the high variance of production serverless environments. To test generalization, we evaluate the trained policy zero-shot on three held-out workloads (Workload 1–3): a high-traffic noon period

Table 3: Summary of training and test workloads.

Workload	Period	Invocations	Generalization
Training	Day01, midnight	8,285	–
Workload 1	Day02, midnight	8,627	day-to-day
Workload 2	Day01, noon	9,463	time-of-day
Workload 3	Day10, evening	18,155	time-of-day & day-to-day
Workload 4	Day02, midnight	8,627	heterogeneous
Workload 5	Day01, noon	9,463	heterogeneous
Workload 6	Day10, evening	18,155	heterogeneous

of Day 1 (intra-day shift), the first two minutes of Day 2 (inter-day shift), and a four-minute period of Day 10 (both), in Table 3.

To check that our results are not tied to a single function implementation, we build three heterogeneous workloads (Workload 4–6) spanning the resource dimensions from prior characterization studies (e.g., SeBS [9]): CPU-bound (recursive Fibonacci, our calibrated baseline), memory-bandwidth-bound (sequential array scan/copy/write), and I/O-blocking (a short nanosleep with cached file read and checksum). The three stress different scheduling paths: CPU-bound tasks drive time-slice and priority accounting; memory-bound tasks stay runnable but have contention-inflated service times; and I/O-blocking tasks are the only ones that transition between runnable and blocked states. We mix them 70%/20%/10%, so most load exercises the CPU-bound behavior our scheduler targets while the memory and I/O fractions stress-test service-time uncertainty and blocked-task transitions. Our design has no dedicated I/O mechanism, but blocked tasks still benefit indirectly: on wakeup, load balancing can place them on a less busy core to resume sooner.

Because serverless platforms continuously execute function invocations, we characterize scheduling performance by two metrics accumulated over a workload. For each invocation, its *latency* is the time from arrival to when it first runs, and its *runtime* is the time from its first run to completion; we report *accumulated task latency* and *accumulated task runtime* as the sums over all invocations in the workload, for both of which lower is better. Accumulated latency is critical for deadline-constrained, user-facing workloads, whereas accumulated runtime is directly related to billed cost under the pay-as-you-go pricing model; we report cost as accumulated runtime normalized to CFS.

6 EVALUATION

We evaluate the schedulers synthesized by our system along three axes: effectiveness, generality, and the runtime cost of the synthesis architecture. We first establish that the learned policies hold up beyond the conditions they were trained on: Section 6.1 shows they remain effective when the workload distribution shifts across intra-day and inter-day traffic patterns, and Section 6.2 shows the frozen models transfer to heterogeneous workloads without re-training. We then characterize the resulting behavior and its price: Section 6.3 analyzes task-completion throughput, and Section 6.4 quantifies the per-task scheduling overhead, confirming the approach is practical on the decision path. Finally, Section 6.5 turns to our central claim: by tuning the preemption parameters at deployment time, our schedulers with learned task placement policy span the latency–runtime trade-off, and we use this view to examine the provider–client cost trade-off and the fairness–latency trade-off across schedulers.

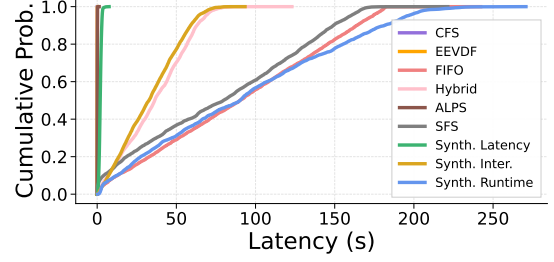


Figure 6: CDFs of response latency for a workload extracted from the Microsoft Azure trace under CFS, EEVDF, FIFO, Hybrid [75], ALPS [16], SFS [15], and three synthesized scheduling policies on 50 CPU cores. Synthesized Latency closely tracks the low-latency schedulers (CFS, EEVDF, ALPS), showing it stays in its target region on an unseen workload.

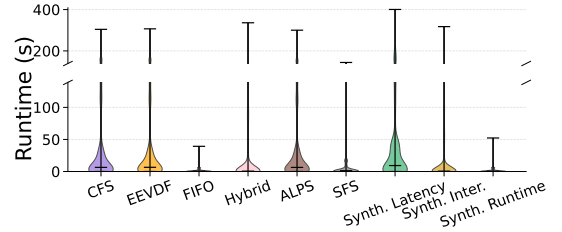


Figure 7: Task runtime distributions for a workload extracted from the Microsoft Azure trace under CFS, EEVDF, FIFO, Hybrid [75], ALPS [16], SFS [15], and three synthesized scheduling policies on 50 CPU cores. Synthesized Runtime overlaps FIFO, the practical runtime lower bound.

6.1 Generalization to Unseen Arrival Pattern

We first evaluate the robustness of our learned schedulers under workload shifts and previously unseen arrival patterns. Because each scheduler is synthesized from a fixed set of training workloads, a key question is whether the learned policy remains effective once the workload distribution changes at deployment time.

In RL evaluation, the trained policy is executed without exploration (i.e., $\epsilon = 0$ in ϵ -greedy), and performance metrics such as total accumulated latency and total runtime are collected. We evaluate our synthesized scheduler on unseen test workloads sampled from different time windows of the Azure trace. These workloads represent various real-world traffic patterns, including intra-day shifts (e.g., noon vs. midnight) and inter-day variations (e.g., Day 1 vs. Day 2); see Section 5.2 for details.

Figures 6 and 7 report results for Workload 1, one of the held-out test workloads defined in Section 5.2, using policies trained on the training workload from the same Azure trace. For this workload, we plot the CDF of per-task response latency and the distribution of per-task runtime across all schedulers. Despite the workload shift, each synthesized policy stays within its intended operating region. In the response-latency CDF, Synthesized Latency clusters with the low-latency schedulers (CFS, EEVDF, ALPS), completing nearly all tasks within a few seconds and staying far ahead of the higher-latency schedulers (Hybrid, FIFO, SFS). In the runtime distribution, Synthesized Runtime overlaps FIFO, the practical runtime

Table 4: Performance comparison on heterogeneous workloads (Workload 4–6). Latency and runtime are accumulated values across all tasks. Cost is normalized to CFS within each workload. Lower is better for all metrics. Each synthesized policy holds its target operating region. Values are mean $\pm$ standard deviation over 5 independent runs.

Policy	Latency (s)	Runtime (s)	Cost
<i>Workload 4</i>			
Low-latency region			
CFS	173 $\pm$ 2	209,279 $\pm$ 2,247	1.0000
EEVDF	168 $\pm$ 0.4	212,022 $\pm$ 889	1.0132 $\pm$ 0.0104
ALPS	567 $\pm$ 6	204,328 $\pm$ 1,594	0.9765 $\pm$ 0.0156
Synthesized Latency	14,524 $\pm$ 668	308,601 $\pm$ 11,137	1.4748 $\pm$ 0.0592
Intermediate region			
Hybrid	219,258 $\pm$ 1,151	152,015 $\pm$ 881	0.7264 $\pm$ 0.0069
Synthesized Inter.	194,694 $\pm$ 3,485	143,681 $\pm$ 750	0.6866 $\pm$ 0.0081
Low-runtime region			
FIFO	631,506 $\pm$ 5,694	75,814 $\pm$ 601	0.3623 $\pm$ 0.0056
SFS	541,086 $\pm$ 19,282	84,253 $\pm$ 1,862	0.4027 $\pm$ 0.0123
Synthesized Runtime	657,675 $\pm$ 7,182	69,184 $\pm$ 914	0.3306 $\pm$ 0.0059
<i>Workload 5</i>			
Low-latency region			
CFS	252 $\pm$ 1	323,220 $\pm$ 1,110	1.0000
EEVDF	247 $\pm$ 2	324,492 $\pm$ 2,471	1.0039 $\pm$ 0.0070
ALPS	833 $\pm$ 5	313,711 $\pm$ 1,018	0.9706 $\pm$ 0.0049
Synthesized Latency	16,288 $\pm$ 646	402,266 $\pm$ 6,471	1.2445 $\pm$ 0.0168
Intermediate region			
Hybrid	325,889 $\pm$ 1,980	242,447 $\pm$ 1,250	0.7501 $\pm$ 0.0061
Synthesized Inter.	284,653 $\pm$ 2,091	225,268 $\pm$ 881	0.6970 $\pm$ 0.0042
Low-runtime region			
FIFO	985,991 $\pm$ 4,176	110,750 $\pm$ 587	0.3426 $\pm$ 0.0011
SFS	870,321 $\pm$ 12,469	118,430 $\pm$ 1,116	0.3664 $\pm$ 0.0034
Synthesized Runtime	1,022,983 $\pm$ 18,546	100,151 $\pm$ 2,695	0.3098 $\pm$ 0.0080
<i>Workload 6</i>			
Low-latency region			
CFS	733 $\pm$ 12	764,200 $\pm$ 3,537	1.0000
EEVDF	732 $\pm$ 4	766,990 $\pm$ 2,212	1.0037 $\pm$ 0.0070
ALPS	2,391 $\pm$ 6	732,933 $\pm$ 2,330	0.9591 $\pm$ 0.0038
Synthesized Latency	41,778 $\pm$ 2,076	940,402 $\pm$ 19,288	1.2305 $\pm$ 0.0226
Intermediate region			
Hybrid	925,879 $\pm$ 3,522	522,586 $\pm$ 4,468	0.6838 $\pm$ 0.0061
Synthesized Inter.	762,273 $\pm$ 44,265	496,935 $\pm$ 8,440	0.6503 $\pm$ 0.0125
Low-runtime region			
FIFO	2,556,550 $\pm$ 34,911	271,454 $\pm$ 2,397	0.3552 $\pm$ 0.0023
SFS	2,294,877 $\pm$ 46,732	279,546 $\pm$ 9,007	0.3658 $\pm$ 0.0131
Synthesized Runtime	2,559,328 $\pm$ 26,399	241,361 $\pm$ 5,444	0.3158 $\pm$ 0.0077

lower bound. Between these two extremes, Synthesized Intermediate (selected in Section 4.5) balances the two objectives, trading a moderate amount of latency for substantially lower runtime.

6.2 Heterogeneous Workload

To test whether the learned policies transfer beyond a single function type, we evaluate the frozen models, without retraining, on three heterogeneous workloads (Workload 4–6) that mix CPU-bound, memory-bandwidth-bound, and I/O-blocking functions (Section 5.2). All three synthesized policies retain their target operating regions on these mixed workloads, confirming that the learned behavior is not specific to the original CPU-bound Fibonacci implementation. Table 4 quantifies this behavior. The lowest accumulated task latency comes from the low-latency schedulers CFS, EEVDF, and ALPS, which Synthesized Latency does not match. Against every other scheduler (Hybrid, FIFO, SFS), however, Synthesized Latency reduces accumulated latency by more than an order of magnitude on all three workloads. It thus stays in the low-latency region, at the cost of higher runtime than CFS, EEVDF, and ALPS.

Table 5: Function completion timing across scheduling policies (0–120s / 0–240s: share completed within each window; Tail >240s: share finishing after 240s; $T_{99\%}$: time to complete 99%). Synthesized policies stay comparable to the baselines on completion throughput.

Policy	0–120s	0–240s	Tail >240s	$T_{99\%}$
CFS	69.3%	89.2%	10.8%	359.2s
EEVDF	69.2%	89.4%	10.6%	361.5s
FIFO	33.3%	66.0%	34.0%	358.5s
Hybrid	48.6%	88.7%	11.3%	422.1s
ALPS	69.7%	89.8%	10.2%	352.1s
SFS	36.5%	67.8%	32.2%	351.9s
Synthesized Latency	66.4%	85.9%	14.1%	413.2s
Synthesized Intermediate	50.9%	90.2%	9.8%	361.9s
Synthesized Runtime	32.8%	66.4%	33.6%	377.0s

Synthesized Intermediate dominates the Hybrid heuristic it targets on all three workloads, cutting accumulated task latency by 11% to 18% and accumulated task runtime and cost by 5% to 7%. Synthesized Runtime reaches the low-runtime region, matching FIFO on both accumulated task runtime and cost.

Although non-preemptive FIFO is expected to minimize accumulated task runtime, this expectation holds only for tasks that run to completion without blocking after their first dispatch. Decomposing accumulated task runtime by function type confirms this distinction: for continuously runnable CPU- and memory-bandwidth-bound tasks, FIFO achieves 2–11% lower accumulated task runtime than Synthesized Runtime. I/O-blocking tasks, in contrast, repeatedly block on read operations and must be re-dispatched after each wakeup. Under FIFO, a woken task is requeued at the tail and, when the system is oversubscribed, waits behind other work before it can resume. On the I/O-blocking tasks, this waiting makes FIFO’s first-run-to-completion times 12–15% longer than Synthesized Runtime’s, which places and rebalances woken tasks on less-loaded cores. Since I/O-blocking tasks account for roughly 79% of accumulated task runtime, this reduction outweighs FIFO’s small advantage on the non-blocking tasks, allowing Synthesized Runtime to achieve lower total accumulated runtime and cost.

6.3 Throughput

In this section, we analyze task completion throughput, measured by the number of tasks completed over time. Table 5 summarizes the completion-time distribution across scheduling policies. CFS, EEVDF, and ALPS complete a large fraction of tasks within the first 120 seconds, around 69%, and have short tails: only about 10% of tasks complete after 240 seconds. In contrast, FIFO completes only 33% of tasks within 120 seconds and has a much longer tail, with 34% of tasks completing after 240 seconds. The latency-optimized synthesized scheduler closely follows these low-latency schedulers over most of the distribution. It completes 66% of tasks within 120 seconds and 86% within 240 seconds, substantially ahead of the FIFO-style schedulers (FIFO, SFS, and Synthesized Runtime), which complete only about one third of tasks within 120 seconds. The latency-optimized synthesized scheduler’s tail is slightly longer than the best low-latency baseline: 14% of tasks complete after 240 seconds, and 99% complete by roughly 413 seconds, compared with about 352 seconds for ALPS. The intermediate synthesized scheduler, which targets the same operating region as the Hybrid heuristic, tracks the low-latency schedulers across the distribution:

Table 6: Per-task scheduling overhead of the synthesized schedulers, split into scheduler-agent communication and RL inference. Overhead of μ s scale is sufficiently small for serverless scheduling decisions.

	Comm	RL inference	Total
Scheduling overhead per task (μ s)	0.67	134.50	135.17
Share of whole workload (%)	0.002	0.40	0.40

it completes 51% of tasks within 120 seconds and 90% within 240 seconds, with a tail (10%) and $T_{99\%}$ (362 seconds) comparable to CFS, EEVDF, and ALPS. The runtime-optimized synthesized scheduler exhibits a completion distribution similar to FIFO. Since it optimizes accumulated task runtime, the learned policy uses longer preemption time slices and performs fewer preemptions, making it behave similarly to non-preemptive FIFO.

6.4 Overhead Analysis

Table 6 reports the per-task scheduling overhead of the synthesized schedulers on the sched_ext backend, decomposed into scheduler-agent communication (Comm) and RL inference. The total per-task overhead is about 135 μ s and is dominated by RL inference (134.50 μ s/task), while communication between the scheduler and the RL agent adds only 0.67 μ s/task. Aggregated over an entire episode, this amounts to just 0.40% of the whole workload’s runtime (0.002% for communication), as the agent runs on a single dedicated core alongside the 50 worker cores. The per-task cost is thus small enough to support online scheduling decisions for serverless functions, whose runtimes are typically on the order of milliseconds to seconds.

We further evaluate whether GPU execution reduces the online inference overhead of the synthesized scheduler. This experiment only changes the PyTorch execution device while keeping the same trained weights, model architecture, batch size, and torch build. We run a controlled per-decision microbenchmark on an idle agent with batch size one, matching the inference pattern used during evaluation. The GPU setting uses an NVIDIA A30, while the CPU setting disables CUDA. Table 7 shows that GPU execution increases inference overhead for both decision paths. CPU-core assignment is 2.57 $\times$ slower on the GPU, and preemption time-slice updates are 2.98 $\times$ slower. The per-task scheduling overhead follows the CPU-core assignment path because each task requires one placement inference, while the global time-slice update is amortized across tasks. Communication remains unchanged because transferring the scheduler state to the RL agent and returning the selected action do not depend on the inference device. These results indicate that GPU execution is not beneficial for our online scheduler: the policy networks are small and each decision uses a single state vector, so kernel-launch, host-device transfer, and synchronization overhead dominate the computation. Therefore, no extra costly hardware is needed for performing inference during serverless scheduling.

6.5 Latency-Cost Trade-off in Practice

In our serverless workload setting, the Pareto space is defined by two objectives: accumulated task latency and accumulated task runtime. Accumulated latency is critical for deadline-constrained workloads, whereas accumulated runtime directly determines user cost under the pay-as-you-go pricing model. Reducing latency often

Table 7: GPU/CPU overhead ratio for scheduler decision paths on an NVIDIA A30. Values larger than one indicate that GPU execution is slower, so no accelerator is needed for online inference.

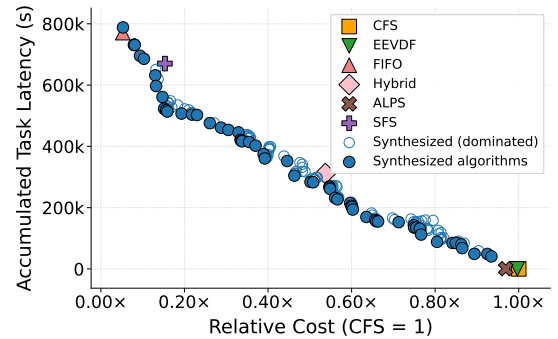
Metric	Comm	RL inference	Total
CPU core assignment request	1.00	2.57	2.57
Preemption time-slice update	1.00	2.98	2.98
Scheduling overhead per task	1.00	2.57	2.57

requires aggressive scheduling decisions, such as earlier preemption, which can increase total runtime, as observed in CFS, EEVDF, and ALPS [16]. Conversely, minimizing accumulated runtime can reduce user-facing cost but may adversely affect deadline-constrained tasks, as demonstrated by FIFO and SFS [15]. Therefore, a single-objective evaluation cannot fully characterize scheduler behavior.

The Pareto space provides a unified view of the latency-runtime trade-off. Each point on the Pareto frontier represents a feasible operating point where improving one objective necessarily degrades the other. This characterization is particularly relevant to serverless environments, where applications exhibit diverse priorities: deadline-constrained workloads favor low-latency operating points, whereas cost-sensitive workloads prefer low-runtime ones.

Figure 8 shows the latency-cost trade-off, where user cost is derived from accumulated task runtime. CFS lies in the low-latency, high-cost region, whereas FIFO reduces cost at the expense of higher accumulated task latency. By evaluating the synthesized scheduler under different preemption configurations, we obtain a set of operating points spanning these two extremes. A point is dominated if some other synthesized point is at least as good on both latency and cost, and better on one. We draw these as hollow markers; the solid points form the empirical Pareto frontier. These points trace a latency-cost trade-off curve spanning the region between the FIFO-like and CFS-like extremes, letting system operators pick an operating point that matches their latency targets and cost budgets in practical real-world FaaS scenarios.

Figure 9 plots provider headroom against client cost improvement, both measured relative to CFS. Provider headroom is defined by the amount of CPU time that the respective scheduling policy can clear compared to CFS. Effectively, provider headroom is a measure of how much more workload the cloud could pack on a single machine, increasing profit for the cloud provider. Each point

**Figure 8: Latency-cost trade-off. CFS, EEVDF, and ALPS [16] achieve low latency with high cost, while FIFO and SFS [15] reduce cost but with higher latency. Hybrid [75] provides intermediate baseline. Our synthesized schedulers span the trade-off space.**

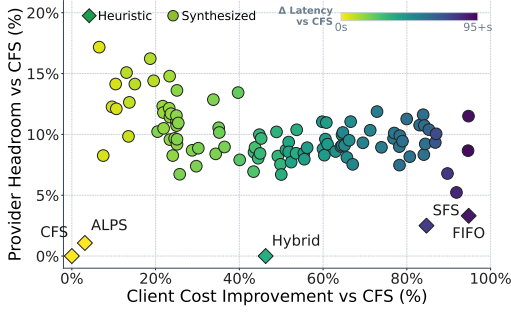


Figure 9: Provider headroom versus client cost improvement, both relative to CFS. Synthesized schedulers can increase profit both for provider and client, depending on latency.

in the figure is a policy evaluated on the same workload, the color indicates the mean per-task latency increase over CFS (light for low, dark for high). EEVDF and ALPS offer no improvements, neither for clients nor for providers.

The state-of-the-art schedulers address a single objective: FIFO attains the largest cost improvement but at the highest latency and near-zero headroom, while Hybrid [75] attains cost improvement but leaves no provider headroom. In contrast, the synthesized algorithms occupy the upper region, jointly achieving high headroom and cost improvement, with several of them keeping latency low. Overall, by exploring the empirical Pareto front of the trade-off space using synthesized schedulers we find interesting points which improve the profitability of both clients (barring increased latency), and cloud providers, allowing them to bin-pack more workloads onto the same resources.

To ensure that the synthesized schedulers offer trade-offs in the latency-runtime space without unfairly starving tasks, we perform a Gini coefficient analysis [51]. Figure 10 plots the wait-time Gini coefficient together with the mean task latency across all schedulers; a lower wait-time Gini indicates a more even distribution of waiting time across tasks, and the color represents mean task latency. The synthesized schedulers span a broad range of operating points, and their wait-time Gini stays within the same range as the heuristic schedulers. This shows that our synthesized schedulers offer fairness comparable to existing schedulers, and therefore do not obtain their latency-runtime trade-offs by starving tasks.

7 RELATED WORK

In this article, we proposed learning-based OS serverless scheduling. We summarize related work in learning-based resource management, learning-based methods for serverless platforms, serverless-aware OS-level scheduling, and hardware-aware serverless.

Learning-based resource management. Learning-based methods have been applied to a wide range of systems problems, including network congestion control [1, 26, 38], resource allocation and partitioning [6, 45, 47, 56, 69], autoscaling [19, 32, 61, 67, 72], and cluster scheduling [24, 46, 55, 73]. These demonstrate the value of learning and adaptation for systems decision making. However, they operate at the network, cluster, job, or resource-partitioning level, rather than at the OS scheduling layer of a serverless worker.

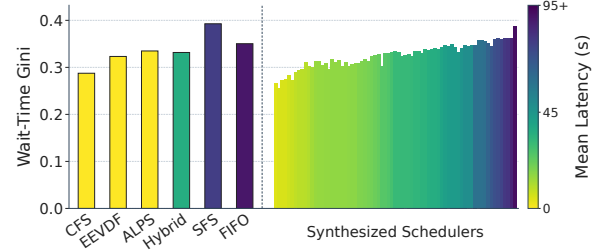


Figure 10: Wait-time fairness and mean task latency across heuristic and synthesized schedulers according to the Gini coefficient. The synthesized schedulers offer similar fairness to existing schedulers.

Learning-based methods for serverless platforms. Recent work has also applied learning-based methods to serverless platforms [44, 57, 58, 71]. These systems focus on platform-level function scheduling, resource provisioning, autoscaling, or hybrid serverless/serverful placement. In contrast, our work focuses on the OS-level scheduling on a worker node. We study how learning can synthesize OS-level scheduling policies for core assignment, preemption, and load rebalancing on a multi-core server.

Serverless-aware OS-level scheduling. Several systems optimize serverless execution through OS-level scheduling mechanisms [2, 15, 16, 30, 31, 75]. These systems improve over general-purpose schedulers by using mechanisms such as centralized dispatch, locality-aware placement, preemption, work stealing, or hybrid scheduling. Most of these approaches, however, rely on fixed heuristic policies. Our work uses a reinforcement learning architecture to synthesize schedulers that can be tuned toward different points in the latency-runtime trade-off space.

Hardware- and data-aware serverless optimization. Beyond CPU scheduling policies, prior work has explored hardware- and data-aware serverless optimization, including memory-bandwidth and cache-aware scheduling [18, 52, 63], local memory, container caching, and edge object-store mechanisms [7, 37, 70], Data Processing Units (DPUs) [42], and hardware-assisted preemption [36]. These techniques provide finer control over serverless execution resources and data locality. Our work focuses on a complementary direction: synthesizing tunable OS-level scheduling policies for CPU core assignment, preemption, and rebalancing.

8 CONCLUSION

We presented a class of synthesized schedulers for the execution of serverless functions. Rather than relying on fixed heuristics, we synthesize scheduling policies using a reinforcement learning architecture that jointly learns core assignment, preemption, and rebalancing under different reward objectives, allowing synthesized schedulers to be tuned to achieve a near-continuous set of operating points spanning the latency-runtime trade-off space. We implemented our schedulers on top of the Linux extensible scheduling framework `sched_ext` and evaluated them on real-world Azure function traces. Our results show that synthesized schedulers are competitive with time-tested schedulers such as CFS, EEVDF, and FIFO, and can be tuned to span operating points across the latency-runtime space, whereas each existing scheduler remains confined to a narrow region of the trade-off space. We show that synthesized

schedulers can reduce user cost while improving platform profitability. More broadly, our results suggest that synthesizing schedulers is a practical path toward workload-specialized OS scheduling. We release the source code at [Anonymous Link].

REFERENCES

- [1] Soheil Abbasloo, Chen-Yu Yen, and H Jonathan Chao. 2020. Classic meets modern: A pragmatic learning-based congestion control for the internet. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*. 632–647.
- [2] Mania Abdi, Samuel Ginzburg, Xiayue Charles Lin, Jose Faleiro, Gohar Irfan Chaudhry, Inigo Goiri, Ricardo Bianchini, Daniel S Berger, and Rodrigo Fonseca. 2023. Palette load balancing: Locality hints for serverless functions. In *Proceedings of the eighteenth European conference on computer systems*. 365–380.
- [3] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. 2020. Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX symposium on networked systems design and implementation (NSDI 20)*. 419–434.
- [4] AWS. 2025. AWS Lambda pricing. <https://aws.amazon.com/lambda/pricing/>. [Online; accessed 10-July-2025].
- [5] Marc Brooker, Mike Danilov, Chris Greenwood, and Phil Piwonka. 2023. On-demand container loading in {AWS} lambda. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. 315–328.
- [6] Ruobing Chen, Jinping Wu, Haosen Shi, Yusen Li, Xiaoguang Liu, and Gang Wang. 2021. DRLPart: A deep reinforcement learning framework for optimally efficient and robust resource partitioning on commodity servers. In *Proceedings of the 30th International Symposium on High-Performance Parallel and Distributed Computing*. 175–188.
- [7] Xin Chen, Manoj Prabhakar Paidiparthi, Chen Qian, and Liting Hu. 2025. Copybara: an Edge-Friendly Distributed Object Store for Diverse Serverless Functions. In *Proceedings of the 26th International Middleware Conference*. 31–44.
- [8] Completely Fair Scheduler. 2026. <https://docs.kernel.org/scheduler/sched-design-CFS.html>. [Online; accessed 10-July-2026].
- [9] Marcin Copik, Grzegorz Kwasniewski, Maciej Besta, Michal Podstawski, and Torsten Hoeftler. 2021. Sebs: A serverless benchmark suite for function-as-a-service computing. In *Proceedings of the 22nd International Middleware Conference*. 64–78.
- [10] Dmitry Duplyakin, Robert Ricci, Aleksander Maricq, Gary Wong, Jonathon Duerig, Eric Eide, Leigh Stoller, Mike Hibler, David Johnson, Kirk Webb, et al. 2019. The design and operation of {CloudLab}. In *2019 USENIX annual technical conference (USENIX ATC 19)*. 1–14.
- [11] Earliest Eligible Virtual Deadline First. 2026. <https://docs.kernel.org/scheduler/sched-eevdf.html>. [Online; accessed 10-July-2026].
- [12] Adam Eivy and Joe Weinman. 2017. Be wary of the economics of “serverless” cloud computing. *IEEE Cloud Computing* 4, 2 (2017), 6–12.
- [13] Tarek Elgamal, Atul Sandur, Klara Nahrstedt, and Gul Agha. 2018. Costless: Optimizing cost of serverless computing through function fusion and placement. In *2018 IEEE/ACM Symposium on Edge Computing (SEC)*. IEEE, 300–312.
- [14] Vincent Francois-Lavet, Peter Henderson, Riashat Islam, Marc G. Bellemare, and Joelle Pineau. 2018. An introduction to deep reinforcement learning. *Foundations and trends® in machine learning* 11, 3-4 (2018), 219–354.
- [15] Yuqi Fu, Li Liu, HaoLiang Wang, Yue Cheng, and Songqing Chen. 2022. SFS: Smart OS scheduling for serverless functions. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–16.
- [16] Yuqi Fu, Ruizhe Shi, HaoLiang Wang, Songqing Chen, and Yue Cheng. 2024. {ALPS}: An Adaptive Learning, Priority {OS} Scheduler for Serverless Functions. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. 19–36.
- [17] Scott Fujimoto, Herke van Hoof, and David Meger. 2018. Addressing function approximation error in actor-critic methods. In *International conference on machine learning*. PMLR, 1587–1596.
- [18] Rastko Gajanin, Cynthia Marcelino, and Stefan Nastic. 2025. Performance Isolation for Serverless Functions. *IEEE Transactions on Services Computing* (2025).
- [19] Yisel Garí, David A Monge, Elna Pacini, Cristian Mateos, and Carlos García Garino. 2021. Reinforcement learning-based application autoscaling in the cloud: A survey. *Engineering Applications of Artificial Intelligence* 102 (2021), 104288.
- [20] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. 2018. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*. Pmlr, 1861–1870.
- [21] Joseph M Hellerstein, Jose Faleiro, Joseph E Gonzalez, Johann Schleier-Smith, Vikram Sreekanti, Alexey Tumanov, and Chenggang Wu. 2018. Serverless computing: One step forward, two steps back. *arXiv preprint arXiv:1812.03651* (2018).
- [22] Al Amjad Tawfiq Isstaif and Richard Mortier. 2023. Towards latency-aware linux scheduling for serverless workloads. In *Proceedings of the 1st Workshop on Serverless Systems, Applications and Methodologies*. 19–26.
- [23] Rishabh Iyer, Musa Unal, Marios Kogias, and George Candea. 2023. Achieving microsecond-scale tail latency efficiently with approximate optimal scheduling. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 466–481.
- [24] Akshay Jajoo, Y Charlie Hu, Xiaojun Lin, and Nan Deng. 2022. Slearn: A case for task sampling based learning for cluster job scheduling. *IEEE Transactions on Cloud Computing* 11, 3 (2022), 2664–2680.
- [25] Jake Brutlag. 2009. <https://research.google/blog/speed-matters/>. [Online; accessed 10-July-2026].
- [26] Nathan Jay, Noga Rotman, Brighton Godfrey, Michael Schapira, and Aviv Tamar. 2019. A deep reinforcement learning perspective on internet congestion control. In *International Conference on Machine Learning*. PMLR, 3050–3059.
- [27] Eric Jonas, Johann Schleier-Smith, Vikram Sreekanti, Chia-Che Tsai, Anurag Khandelwal, Qifan Pu, Vaishaal Shankar, Joao Carreira, Karl Krauth, Neeraja Yadwadkar, et al. 2019. Cloud programming simplified: A berkeley view on serverless computing. *arXiv preprint arXiv:1902.03383* (2019).
- [28] Artjom Joosen, Ahmed Hassan, Martin Asenov, Rajkarn Singh, Luke Darlow, Jianfeng Wang, and Adam Barker. 2023. How does it function? characterizing long-term trends in production serverless workloads. In *Proceedings of the 2023 ACM Symposium on Cloud Computing*. 443–458.
- [29] Kostis Kaffes, Timothy Chong, Jack Tigar Humphries, Adam Belay, David Mazieres, and Christos Kozyrakis. 2019. Shinjuku: Preemptive Scheduling for {μsecond-scale} Tail Latency. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. 345–360.
- [30] Kostis Kaffes, Neeraja J Yadwadkar, and Christos Kozyrakis. 2019. Centralized core-granular scheduling for serverless functions. In *Proceedings of the ACM symposium on cloud computing*. 158–164.
- [31] Kostis Kaffes, Neeraja J Yadwadkar, and Christos Kozyrakis. 2022. Hermod: principled and practical scheduling for serverless functions. In *Proceedings of the 13th Symposium on Cloud Computing*. 289–305.
- [32] Sara Kardani-Moghaddam, Rajkumar Buyya, and Kotagiri Ramamohanarao. 2020. ADRL: A hybrid anomaly-aware deep reinforcement learning-based resource scaling in clouds. *IEEE Transactions on Parallel and Distributed Systems* 32, 3 (2020), 514–526.
- [33] Martin Kleppmann. 2017. *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. “O’Reilly Media, Inc.”.
- [34] Ricardo Koller and Dan Williams. 2017. Will serverless end the dominance of linux in the cloud?. In *Proceedings of the 16th Workshop on Hot Topics in Operating Systems*. 169–173.
- [35] Suyi Li, Wei Wang, Jun Yang, Guangzhen Chen, and Daohe Lu. 2023. Golgi: Performance-aware, resource-efficient function scheduling for serverless computing. In *Proceedings of the 2023 ACM Symposium on Cloud Computing*. 32–47.
- [36] Yueying Li, Nikita Lazarev, David Koufaty, Tenny Yin, Andy Anderson, Zhiru Zhang, G Edward Suh, Kostis Kaffes, and Christina Delimitrou. 2024. Libpreemptible: Enabling fast, adaptive, and hardware-assisted user-space scheduling. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 922–936.
- [37] Zijun Li, Yushi Liu, Linsong Guo, Quan Chen, Jiagan Cheng, Wenli Zheng, and Minyi Guo. 2022. Faasflow: Enable efficient workflow execution for function-as-a-service. In *Proceedings of the 27th acm international conference on architectural support for programming languages and operating systems*. 782–796.
- [38] Xudong Liao, Han Tian, Chaoliang Zeng, Xinchun Wan, and Kai Chen. 2024. Astraea: Towards Fair and Efficient Learning-based Congestion Control. In *Proceedings of the Nineteenth European Conference on Computer Systems*. 99–114.
- [39] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. 2016. Continuous control with deep reinforcement learning. In *4th International Conference on Learning Representations (ICLR)*.
- [40] Changyuan Lin, Yuanzhi Ma, and Mohammad Shahradd. 2026. Demystifying Serverless Costs on Public Platforms: Bridging Billing, Architecture, and OS Scheduling. (2026).
- [41] John DC Little. 1961. A proof for the queuing formula: $L = \lambda W$. *Operations research* 9, 3 (1961), 383–387.
- [42] Guowei Liu, Laiping Zhao, Yiming Li, Zhaolin Duan, Sheng Chen, Yitao Hu, Zhiyuan Su, and Wenyu Qu. 2024. Fuyao: Dpu-enabled direct data transfer for serverless computing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 431–447.
- [43] Jean-Pierre Lozi, Baptiste Lepers, Justin Funston, Fabien Gaud, Vivien Quéma, and Alexandra Fedorova. 2016. The Linux scheduler: a decade of wasted cores. In *Proceedings of the Eleventh European Conference on Computer Systems*. 1–16.
- [44] Anupama Mampage, Shanika Karunasekera, and Rajkumar Buyya. 2025. Deep Reinforcement Learning for Scheduling Applications in Serverless and Serverful Hybrid Computing Environments. *IEEE Transactions on Services Computing* (2025).
- [45] Hongzi Mao, Mohammad Alizadeh, Ishai Menache, and Srikanth Kandula. 2016. Resource management with deep reinforcement learning. In *Proceedings of the 15th ACM workshop on hot topics in networks*. 50–56.

- [46] Hongzi Mao, Malte Schwarzkopf, Shailesh Bojja Venkatakrishnan, Zili Meng, and Mohammad Alizadeh. 2019. Learning scheduling algorithms for data processing clusters. In *Proceedings of the ACM special interest group on data communication*. 270–288.
- [47] Weichao Mao, Haoran Qiu, Chen Wang, Hubertus Franke, Zbigniew Kalbarczyk, Ravishankar Iyer, and Tamer Basar. 2022. A mean-field game approach to cloud resource management with function approximation. *Advances in Neural Information Processing Systems* 35 (2022), 36243–36258.
- [48] Andreas Merkel and Frank Bellosa. 2008. Memory-aware Scheduling for Energy Efficiency on Multicore Processors. *HotPower* 8 (2008), 123–130.
- [49] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. 2013. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602* (2013).
- [50] Nima Nasiri, Nalin Munshi, Simon Daniel Moser, Marius Pirvu, Vijay Sundaresan, Daryl Maier, Thatta Premnath, Norman Böwing, Sathish Gopalakrishnan, and Mohammad Shahrad. 2026. In-production characterization of an open source serverless platform and new scaling strategies. In *Proceedings of the 21st European Conference on Computer Systems*. 2054–2072.
- [51] Włodzimierz Ogryczak and Grzegorz Zalewski. 2019. Fair resource allocation by Gini Index minimization. In *Operations Research Proceedings 2018: Selected Papers of the Annual International Conference of the German Operations Research Society (GOR), Brussels, Belgium, September 12–14, 2018*. Springer, 77–83.
- [52] Jinsu Park, Seongbeom Park, and Woongki Baek. 2019. Copart: Coordinated partitioning of last-level cache and memory bandwidth for fairness-aware workload consolidation on commodity servers. In *Proceedings of the Fourteenth EuroSys Conference 2019*. 1–16.
- [53] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).
- [54] PyO3. 2026. <https://github.com/pyo3/pyo3>. [Online; accessed 10-July-2026].
- [55] Aurick Qiao, Sang Keun Choe, Suhas Jayaram Subramanya, Willie Neiswanger, Qirong Ho, Hao Zhang, Gregory R Ganger, and Eric P Xing. 2021. Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. In *15th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 21)*.
- [56] Haoran Qiu, Subho S Banerjee, Saurabh Jha, Zbigniew T Kalbarczyk, and Ravishankar K Iyer. 2020. {FIRM}: An intelligent fine-grained resource management framework for {SLO-Oriented} microservices. In *14th USENIX symposium on operating systems design and implementation (OSDI 20)*. 805–825.
- [57] Haoran Qiu, Weichao Mao, Archit Patke, Chen Wang, Hubertus Franke, Zbigniew T Kalbarczyk, Tamer Başar, and Ravishankar K Iyer. 2022. SIMPPO: A scalable and incremental online learning framework for serverless resource management. In *Proceedings of the 13th Symposium on Cloud Computing*. 306–322.
- [58] Haoran Qiu, Weichao Mao, Chen Wang, Hubertus Franke, Alaa Youssef, Zbigniew T Kalbarczyk, Tamer Başar, and Ravishankar K Iyer. 2023. {AWARE}: Automate workload autoscaling with reinforcement learning in production cloud systems. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. 387–402.
- [59] Michael Roberts and John Chapin. 2017. *What is Serverless?* O'Reilly Media, Incorporated.
- [60] sched_ext. 2026. <https://sched-ext.com>. [Online; accessed 10-July-2026].
- [61] Lucia Schuler, Somaya Jamil, and Niklas Kühl. 2021. AI-based resource allocation: Reinforcement learning for adaptive auto-scaling in serverless environments. In *2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. IEEE, 804–811.
- [62] Mohammad Shahrad, Rodrigo Fonseca, Inigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. 2020. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In *2020 USENIX annual technical conference (USENIX ATC 20)*. 205–218.
- [63] Amoghavarsha Suresh and Anshul Gandhi. 2021. Servermore: Opportunistic execution of serverless functions in the cloud. In *Proceedings of the ACM symposium on cloud computing*. 570–584.
- [64] Richard S Sutton and Andrew G Barto. 1998. *Reinforcement learning: An introduction*. Vol. 1. MIT press Cambridge.
- [65] Huangshi Tian, Suyi Li, Ao Wang, Wei Wang, Tianlong Wu, and Haoran Yang. 2022. Owl: Performance-aware scheduling for resource-efficient function-as-a-service cloud. In *Proceedings of the 13th Symposium on Cloud Computing*. 78–93.
- [66] Erwin Van Eyk, Lucian Toader, Sacheendra Talluri, Laurens Versluis, Alexandru Uță, and Alexandru Iosup. 2018. Serverless is more: From paas to present cloud computing. *IEEE Internet Computing* 22, 5 (2018), 8–17.
- [67] Tokala Veni and S Mary Saira Bhanu. 2016. Auto-scale: automatic scaling of virtualised resources using neuro-fuzzy reinforcement learning approach. *International Journal of Big Data Intelligence* 3, 3 (2016), 145–153.
- [68] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, and Nando de Freitas. 2016. Dueling network architectures for deep reinforcement learning. In *International conference on machine learning*. PMLR, 1995–2003.
- [69] Zhe Yang, Phuong Nguyen, Haiming Jin, and Klara Nahrstedt. 2019. MIRAS: Model-based reinforcement learning for microservice resource allocation over scientific workflows. In *2019 IEEE 39th international conference on distributed computing systems (ICDCS)*. IEEE, 122–132.
- [70] Hanfei Yu, Rohan Basu Roy, Christian Fontenot, Devesh Tiwari, Jian Li, Hong Zhang, Hao Wang, and Seung-Jong Park. 2024. Rainbowcake: Mitigating cold-starts in serverless with layer-wise container caching and sharing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 335–350.
- [71] Hanfei Yu, Athirai A Irissappane, Hao Wang, and Wes J Lloyd. 2021. Faasrank: Learning to schedule functions in serverless platforms. In *2021 IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS)*. IEEE, 31–40.
- [72] Anastasios Zafeiropoulos, Eleni Fotopoulou, Nikos Filinis, and Symeon Papavasiliou. 2022. Reinforcement learning-assisted autoscaling mechanisms for serverless computing platforms. *Simulation Modelling Practice and Theory* 116 (2022), 102461.
- [73] Di Zhang, Dong Dai, and Bing Xie. 2022. Schedinspector: A batch job scheduling inspector using reinforcement learning. In *Proceedings of the 31st International Symposium on High-Performance Parallel and Distributed Computing*. 97–109.
- [74] Yuxuan Zhao and Alexandru Uta. 2022. Tiny autoscalers for tiny workloads: Dynamic CPU allocation for serverless functions. In *2022 22nd IEEE international symposium on cluster, cloud and internet computing (CCGrid)*. IEEE, 170–179.
- [75] Yuxuan Zhao, Weikang Weng, Rob van Nieuwpoort, and Alexandru Uta. 2024. In Serverless, OS Scheduler Choice Costs Money: A Hybrid Scheduling Approach for Cheaper FaaS. In *Proceedings of the 25th International Middleware Conference*. 172–184.